

Documentación CDT L^AT_EX

PLANTILLA PARA LA ESPECIFICACIÓN DE PROYECTOS

DOCUMENTACIÓN TÉCNICA

— CDT L^AT_EX — Versión 1.0 —

Escuela Superior de Cómputo
Instituto Politécnico Nacional

2 de marzo de 2026

cdtTheme/portada.png

Índice general

List of Macros

1	Package cdtAnalysis.	5
1.1	\RFItem	6
2	Package cdtPersistencia.	15
2.2	\Terminos	16
2.3	\BusinessRules	16
2.4	\Actores	16
2.5	\Interfaces	16
2.6	\UseCases	16
2.7	\Messages	16
2.8	\OtrosElementos	16
2.9	\load	16
2.10	\cdtLabel	17
2.11	\cdtAddElement	17
2.12	\currentElementId	17
2.13	\currentElementFamily	17
2.14	\refElem	18
2.15	\refIdElem	18
2.16	\referencedElementFamily	18
2.17	\oldValue	18
2.18	\setReferencedElementFamily	18
2.19	\makeReferenciasCruzadas	19
2.20	\getReferences	19
2.21	\getReferencesId	19
2.22	\getElement	19
2.23	\save	19

List of Environments

1 Package cdtAnálisis	5
1.1 \requerimientos	6
1.2 \atributoSoftware	6
1.3 \actor	7
1.4 \UserCase	9
2 Package cdtPersistencia	15

Parte I

Estilos

Parte II

Especificación de Software

CAPÍTULO 1

Package cdtAnalysis

El proposito de este paquete es facilitar la documentación en la etapa de análisis de un proyecto de ingeniería de software, contiene comandos y entornos para la especificación de actores, interfaces de usuario y mensajes.

Las dependencias con relación a otros paquetes se listan a continuación:

`chapterfolder` Paquete destinado a los usuarios que tienen una compleja estructura de carpetas, el paquete es utilizado para poder incluir archivos de otros directorios.

`keystroke` Este paquete provee un conjunto de macros para la representación gráfica de las teclas de un teclado de computadora.

`etoolbox` Proporciona un conjunto de primitivas de e-TeX entre otras herramientas destinadas a la generación de plantillas y comandos/environments de L^AT_EX.

`cdt/Business` ir al capitulo.

`cdt/Control` ir al capitulo.

`cdt/cdtPersistencia` ir al capitulo.

`cdt/cdtBook` ir al capitulo.

1.1. Opciones de Paquete

Las opciones de este paquete permiten filtrar el contenido que se desplegará en el documento PDF al momento de la compilación.

Las distintas categorías de visualización son:

`all` Indica que el elemento se desplegará en todas las vistas.

`admin` Indica que el elemento es de administración del proyecto.

`control` El elemento es usado para control de cambios.

`design` El elemento define aspectos específicos de diseño.

Una vez vistos las distintas categorías de visualización, la opciones que permite este paquete permiten acomodar un filtro para mostrar las distintas categorías. Las opciones del paquete son:

userView Muestra solo los elementos con categoria: all. (Opción por defecto)
adminView Muestra los elementos con categorías: all, admin, control o design.
testView Muestra los elementos: all y design

Este paquete es el que debe ser añadido solo una vez en el archivo principal de L^AT_EX. A continuación se muestran algunos ejemplos de como añadirlo, especificando las opciones

```
\usepackage{cdt/cdtAnalisys}
\usepackage[userView]{cdt/cdtAnalisys}
\usepackage[adminView]{cdt/cdtAnalisys}
\usepackage[testView]{cdt/cdtAnalisys}
```

1.2. Especificación de Requerimientos

El paquete define el entorno `\requerimientos` el cual muestra en una tabla los requerimientos iniciales del sistema. El entorno recibe como argumento opcional el título, en caso de no especificarse su valor es Requerimientos iniciales para el sistema.

```
\begin{requerimientos}
  \RFItem{id}{Nombre}{Descripción}
  \RFItem{id}{Nombre}{Descripción}
  ...
\end{requerimientos}
```

```
\begin{requerimientos}[Título]
  \RFItem{id}{Nombre}{Descripción}
  \RFItem{id}{Nombre}{Descripción}
  ...
\end{requerimientos}
```

El comando `\RFItem` permite añadir un nuevo requerimientos al entorno `\requerimientos`. Este comando recibe como argumentos el identificador de requerimiento, el nombre y la descripción del mismo.

```
\RFItem{id}{Nombre}{Descripción}
```

1.3. Atributos de Software

El paquete define el entorno `\atributoSoftware` el cual ayuda a definir los atributos de software que se deben considerar, en esta definición se incluye el objetivo, definición de métricas, unidad de medida, proceso de medición, y la meta o valores de aceptación.

Este entorno recibe como parámetros tres argumentos, los cuales son el identificador del atributo de software, el tipo, y el nombre. A continuación se muestra un ejemplo de como utilizar este entorno.

```
\begin{atributoDeSoftware}{Id}{Tipo}{Nombre}
  \item[Objetivo:] Descripción
  \item[Definición:] Descripción
  \item[Métrica:] Descripción
  \item[Unidad de medida:] Descripción
  \item[Medición:] Descripción
  \item[Meta:] Descripción
\end{atributoDeSoftware}
```

Nota: La medición debe contener una descripción del procedimiento y ambiente en necesarios para obtener evidencia objetiva sobre la propiedad que se pueda presentar en la Unidad de medida descrita. Esta descripción debe garantizar su reproducción: la repetición del procedimiento, bajo las mismas condiciones descritas, de los mismos artefactos de software, deben producir los mismos resultados.

1.4. Especificación de Actores

El paquete define el entorno `\actor` el cual permite especificar las características de los actores que operarán el sistema. Este entorno recibe como parámetros un argumento opcional y tres argumentos obligatorios. El argumento opcional, debe ser una cadena especial especificada en la sección ?? en el apartado X..., a continuación se muestran dos ejemplos de como utilizar este entorno.

```
\begin{Actor}{id}{Nombre del actor}{descripción}
  \item[Área:] Descripción
  \item[Responsabilidades:] \hfill
    \begin{itemize}
      \item Supervisar ...
      \item Plantear ...
      \item Cuidar ...
    \end{itemize}
  \item[Perfil:] \hfill
    \begin{itemize}
      \item Amplia experiencia en el ramo
      \item Licenciatura como mínimo
    \end{itemize}
  \item[Cantidad:] Decenas
\end{Actor}
```

Nota: Se recomienda especificar la cantidad en decenas, centenas, millares, etc.

```
\begin{Actor}[Autor/Daniel0,Version/1.0,Estado/Ok]{id}{Nombre}{descripción}
  \item[Área:] Descripción
  \item[Responsabilidades:] \hfill
    \begin{itemize}
      \item Supervisar ...
      \item Plantear ...
      \item Cuidar ...
    \end{itemize}
  \item[Perfil:] \hfill
    \begin{itemize}
      \item Amplia experiencia en el ramo
      \item Licenciatura como mínimo
    \end{itemize}
  \item[Cantidad:] Cientos
\end{Actor}
```

1.5. Interfaces

1.6. Casos de Uso

Este paquete también brinda el entorno `\UserCase` el cual ayuda a especificar la interacción que tienen uno o varios actores con el sistema al ocupar una funcionalidad del mismo.

```
\begin{UseCase}[Autor/Daniel0,Version/1.0,Estado/Ok]{Id}{Nombre}{Descripción}

% Atributos de control

\UCsection{Atributos}
\UCitem{Actor}{\refElem{ actor }}
\UCitem{Propósito}{ ... }
\UCitem{Entradas}{ \imprimeEntrada }
\UCitem{Origen}{ Mouse y del teclado. }
\UCitem{Salida}{ ... }
\UCitem{Destino}{ ... }
\UCitem{Precondición}{ ... }
\UCitem{Postcondición}{ ... }
\UCitem{Reglas de Negocio}{ ... }
\UCitem{Errores}{ ... }

\UCitem{Referencia Documental}{ ... }
\UCitem{Datos sensibles}{ ... }

% Atributos de diseño

% Atributos de administración

% Atributos de control

\end{UseCase}
```

Al especificar casos de uso se recomienda usar al menos los atributos del ejemplo, si es necesario de pueden añadir o quitar otros elementos. A continuación se escribe la especificación de los comandos:

`\UCsection` Este comando recibe dos argumentos: el primero es opcional y especifica la categoría de visualización, el siguiente es el título de la nueva sección.

```
\UCsection{Atributos}
```

```
\UCsection[design]{Datos de diseño}
```

`\UCitem` Este comando recibe tres argumentos: el primero (opcional) es la categoría de visualización (por defecto 'all'), el segundo es el atributo que se mostrará y el tercer elemento es la descripción de dicho atributo. A continuación se muestran dos ejemplos.

```
\UCitem{Atributo}{Descripción}
```

```
\UCitem[design]{Atributo}{Descripción}
```

1.6.1. Atributos de Control

Normalmente en los atributos de control se especifican al inicio del caso de uso, los que se muestran por defecto son autor, versión y estado; es buena práctica añadir los atributos del supervisor, último cambio, entre otros.

También al realizar las revisiones se acostumbra realizar una sección que separe los datos (atributos) correspondientes a dicha revisión. A continuación se muestra un ejemplo.

```
\UCitem[control]{Supervisor}{...}
\UCitem[control]{Revisor}{...}
\UCitem[control]{Último cambio}{...}

\UCsection[control]{Revisión número 1}
  \UCitem[control]{Fecha}{...}
  \UCitem[control]{Resultado}{Con correcciones}
  \UCitem[control]{Observaciones}{
    \begin{Titemize}
      \Titem Observacion 1
      \Titem ...
    \end{Titemize}
  }
}
```

1.6.2. Atributos de diseño

En los atributos de diseño se especifican consideraciones específicas del caso de uso que implican restricciones al momento de desarrollar/implementar y realizar las pruebas del caso de uso.

```
\UCsection[design]{Datos de Diseño}

  \UCitem[design]{Disparador}{ ... }
  \UCitem[design]{Condiciones de Término}{ ... }
  \UCitem[design]{Efectos Colaterales}{ ... }
  \UCitem[design]{Suposiciones}{ ... }
  \UCitem[design]{Casos de Prueba}{ ... }
```

1.6.3. Atributos de administración

Los atributos de administración de requerimientos permiten saber que el nivel de entendimiento y complejidad del caso de uso para la administración del proyecto.

```
\UCsection[admin]{Datos de Administración de Requerimiento}

  \UCitem[admin]{Viene de}{ \refIdElem{idUseCase} }
  \UCitem[admin]{Requerimientos Funcionales}{ ... }
  \UCitem[admin]{Requerimientos del Sistema}{ ... }
  \UCitem[admin]{Madurez}{ Alta/Media/Baja }
  \UCitem[admin]{Volatilidad}{ Alta/Media/Baja }
  \UCitem[admin]{Complejidad}{ Alta/Media/Baja }
  \UCitem[admin]{Tamaño}{ Grande/Mediano/Pequeño }
  \UCitem[admin]{Prioridad}{ Alta/Media/Baja }
  \UCitem[admin]{Auditable}{ Si/No }
  \UCitem[admin]{Issues}{ ... }
  \UCitem[admin]{Dificultades}{
    \begin{Titemize}
      \Titem Dificultad 1
      \Titem ...
    \end{Titemize}
  }
}
```

1.7. Mensajes

1.7.1. Variables

1.7.2. Commandos

1.7.3. Environments

Parte III

Persistencia y control de cambios

CAPÍTULO 2

Package cdtPersistencia

El paquete `cdtPersistencia` define un conjunto de comandos internos que permiten guardar la información relevante de la especificación de un proyecto de software para (probablemente) ser explotada posteriormente. Además permite que al referenciar un elemento mediante su identificador aparezca un enlace con el nombre del elemento mientras que se añade una entrada para la referencia hecha en el apartado de referencias cruzadas al final del documento.

Las dependencias con relación a otros paquetes se lista a continuación:

`inputenc` Permite especificar la codificación de los caracteres de entrada (la opción utilizada es `'uft8'`).

`babel` Este paquete administra reglas tipográficas con base en el idioma especificado mediante la opción, el idioma utilizado es el español (`'spanish'`)

`fontenc` Permite al seleccionar la codificación del tipo de fuente. Con la opción `T1` se utiliza una codificación de 8-bits, lo que permite tener hasta 256 caracteres (glifos).

`datatool` Permite crear bases de datos mediante comandos de LaTeX, sobre las cuales se pueden aplicar operaciones de ordenamiento, lectura registro a registro, seleccion a traves de condiciones, entre otras.

`pgfkeys` Proporciona un sistema de administración de llaves flexible, dichas llaves son almacenadas internamente en una estructura de arbol con la finalidad de hacer más rápida su busqueda.

`pgffor` Al igual que el paquete `pgfkeys` forma parte del paquete `pgf`. `Pgffor` define el comando `'foreach'` que permite iterar sobre un arreglo de elementos de tipo clave-valor.

`xstring` Provee un conjunto de macros que permiten realizar distintas operaciones sobre cadenas incluyendo operaciones condicionales dependiendo si la cadena cumple o no un criterio especificado.

`multicol` Define un entorno `'multicols'` el cual acomoda el texto en multiples columnas y balancea la cantidad de texto desplegada en cada columna

2.1. Tipos de Elementos

En el paquete se define 7 tipos de elementos los cuales son almacenados en un arreglo correspondiente al tipo de elemento. Cada vez que se procesa un elemento, este se añade al arreglo correspondiente, arreglos son listados a continuación:

- `\Terminos` Almacena los identificadores de los términos de negocio especificados a lo largo del documento. Los identificadores de terminos de negocio deben ser de la forma `tTerminoNegocio` ó `tTermino`.
- `\BusinessRules` Contiene todos los identificadores de las reglas de negocio especificadas en el documento. Los identificadores deben ser de la forma `BRclave` (por ejemplo `BR-S004` o `BR-N025`)
- `\Actores` Contiene los identificadores de los distintos actores especificados en el documento. Los identificadores para los actores deben ser de la forma `aNombreActor` o `aActor`.
- `\Interfaces` Almacena todos los identificadores de las interfaces especificadas en el documento. Dichos identificadores deben ser de la forma (por ejemplo `IN-DAE-UI1`, `IN-DAE-UI1a`)
- `\UseCases` Almacena los identificadores de todos los casos de uso especificados en el documento. Los identificadores deben tener como subcadena a la sílaba `UI`
- `\Messages` Contiene los identificadores de los distintos mensajes del sistema especificados. Los identificadores de los mensajes deben ser de la forma `MSGclave`
- `\OtrosElementos` Contiene los nombres de las entidades y los atributos de las mismas, de la forma `Entidad` o `Entidad.atributo`. Cualquier elemento que no entre en las otras clasificaciones se guardará por defecto aquí.

2.2. Carga de Elementos

La carga de elementos se realiza mediante el comando `\load`, por motivos de optimización los archivos de persistencia son abiertos en modo lectura unicamente en el inicio del documento. Cada archivo de persistencia corresponde a un tipo de elemento.

```
\load{Family}{DB<suffixname>}
```

El comando recibe dos argumentos, el primero es la familia o abreviación del tipo de elemento, y el segundo es el subfijo del nombre de la base de datos. Este comando es usado unicamente al inicio del documento.

```
\AtBeginDocument{
  \load{BR}{BusinessRules}
  \load{Term}{Terminos}
  \load{Element}{Elements}
  \load{UseCase}{UseCases}
  \load{MSg}{Messages}
  \load{UI}{Interfaces}
  \load{Actor}{Actores}
}
```

2.3. Añadiendo Elementos

Cada vez que se quiere añadir un elemento es necesario indicar establecer un punto para las posibles futuras referencias de ese mismo elemento, para ello se utiliza el siguiente comando.

`\cdtLabel` Al usar este comando se añadirá el punto de referencia a cadena especificada, cuando no se quiera imprimir una cadena entonces el segundo argumento debería estar vacío.

```
\cdtLabel{id}{Nombre}
```

`\cdtAddElement` Este comando permite añadir un elemento al registro de llaves y al arreglo correspondiente del tipo de elemento introducido, especificando el autor, version y estado de dicho elemento

```
\cdtAddElement{id}{Nombre}{Autor}{Version}{Estado}
```

Las variables que son asignadas en este comando son:

`\currentElementId` Almacena el "id" del elemento que se está procesando en cada momento. Dependiendo de la forma que tenga el "id", este se añadirá a los arreglos `\Terminos`, `\Actores`, `\BusinessRules`, `\Messages`, `\Interfaces`, `\UseCases` o `\OtrosElementos`.

`\currentElementFamily` Almacena la familia del elemento que está actualmente procesado, puede contener los valores Term (para Terminos), Actor (para Actores), BR (para Reglas de Negocio), MSG (para Mensajes), UseCase (para Casos de Uso), UI (para interfaces) y Element (para otros elementos).

Las claves y valores que se añaden al registro de claves están dados de la siguiente forma:

```
/\currentElementFamily/id -> Nombre  
/\currentElementFamily/id/author -> Autor  
/\currentElementFamily/id/version -> Version  
/\currentElementFamily/id/status -> Estado
```

A veces es necesario cambiar el identificador del elemento actual que se está procesando, para ello se ocupa el siguiente comando, que sobrescribe a `\currentElementId`

```
\setCurrentElementId{id}
```

2.4. Manejo de Referencias

Cada vez que sea necesario referenciar algún elemento dentro de la especificación de otro, se recomienda usar el comando `\refElem` o `\refIdElem`.

`\refElem` Crea un enlace con el nombre del elemento que es referenciado.

```
\refElem{id}
```

`\refIdElem` Crea un enlace con el identificador y nombre del elemento que es referenciado, de la forma "id:elemento".

```
\refIdElem{id}
```

Cabe resaltar que el paquete maneja internamente un arreglo por cada elemento que es referenciado. Dicho arreglo contiene los identificadores de los elementos que referencian a un elemento en particular.

Por ejemplo: Si un elemento A , es referenciado primero por un elemento B , y posteriormente A es referenciado por un elemento C , entonces el arreglo A_{ref} contiene a B y C .

Nota: El arreglo A_{ref} es guardado en la llave de pgfkeys $/Reference/A$

Para realizar la tarea anteriormente mencionada, se utilizan las siguientes variables:

- `\referencedElementFamily` Variable que contiene la familia de PGFKeys del elemento a referenciar.
- `\oldValue` Variable que almacena temporalmente el arreglo del elemento a referenciar, antes de ser actualizado.
- `\currentElementId` Variable que almacena el identificador del elemento que está siendo especificado actualmente.

`\setReferencedElementFamily` Es el comando encargado de asignarle valor a la variable `\referencedElementFamily`, recibe como parámetro el id del elemento a referenciar.

```
\setReferencedElementFamily{id}
```

2.5. Impresion de Referencias Cruzadas

Este paquete proporciona una macro la cual imprime todas las referencias cruzadas de los distintos elementos este comando es el siguiente:

`\makeReferenciasCruzadas` Imprime las referencias cruzadas de Terminos de Negocio, Actores, Elementos, Reglas de Negocio, Casos de Uso, Mensajes e Interfaces de Usuario.

```
\makeReferenciasCruzadas
```

Dentro de `\makeReferenciasCruzadas` se llama al los comandos internos.

`\getReferences` Itera sobre los arreglos de elementos para imprimir sus referencias cruzadas. Este comando recibe el par de argumentos familia y arreglo, de un tipo de elementos.

```
\getReferences{Familia}{Macro<Array>}
```

```
\getReferences{BR}{\BusinessRules}
```

`\getReferencesId` Provee la misma funcionalidad que `\getReferences` solo que presenta el nombre del elemento principal junto con su identificador, de la forma "id:elemento".

```
\getReferencesId{Familia}{Macro<Array>}
```

Finalmente, para escribir cada entrada en las referencias cruzadas de nn elemento en particular, se utiliza el comando `\getElement` el cual escribe una sentencia dependiendo del tipo de elemento

```
\getElement{id}
```

Las frase que imprime este comando pueden ser: "El termino A", "El actor B", "La regla de negocio BR-N0012", entre otros.

2.6. Persistencia de Elementos

Este paquete brinda la funcionalidad de que automáticamente se guarde en distintas bases de datos los elementos que fueron especificados a lo largo del documento. El comando que realiza esta tarea es:

`\save` Guarda un tipo de elementos en una base de datos en el directorio db, especificando la familia de los elementos, la macro que contiene el arreglo de dichos elementos y el nombre de la base de datos.

```
\save{Familia}{Macro<Array>}{DB<suffix-name>}
```

Por ejemplo, el comando `\save{BR}{\BusinessRules}{BusinessRules}` guardaría las Reglas de negocio, en una base de datos llamada cdtBusinessRules.csv

Nota: Cada registro en las bases de datos tiene como atributos: id, name, author, version, status. Esto con la finalidad de que los datos puedan ser explotados por alguna otra herramienta externa.